

Revision — 2.1.5 Thinking Concurrently

OCR H446 — one-page revision summary

Must know

- **Thinking concurrently** = identifying parts of a problem that can be carried out **at the same time** (in parallel) or interleaved, rather than strictly one after another.
- Only **independent** tasks — those with **no data dependency** — can run concurrently; a task that needs another's output must stay **sequential**.
- **Benefits:** faster completion / higher throughput, better use of **multi-core** hardware, and a more **responsive** UI (interface stays free while a long task runs).
- **Trade-offs:** harder to design, program and debug; risk of **race conditions** and **deadlock**; results must be **synchronised / recombined** afterwards.
- Concurrency is **not always faster**: on a single core it is **time-slicing** (interleaving), not true parallelism; there is scheduling **overhead**; and some problems are inherently sequential.
- **Race condition:** the result depends on the unpredictable **timing** of concurrent tasks sharing data. **Deadlock:** tasks each wait for a resource the other holds, so none proceed.
- Distinguish **parallel processing** (literally simultaneous on multiple cores) from **concurrency via time-slicing** on one core.

Key terms

Term	Definition
Thinking concurrently	Spotting tasks that can run at the same time / interleaved
Data dependency	When one task needs the output of another (must stay sequential)
Parallel processing	Multiple tasks running literally simultaneously on multiple cores
Time-slicing	Rapidly switching between tasks on one core to appear concurrent
Race condition	Result depends on the unpredictable timing of concurrent tasks
Deadlock	Tasks each wait for a resource the other holds → none proceed
Synchronisation	Coordinating concurrent tasks and recombining their results

Worked example / how to — apply concurrency to a weather-forecast app

- **Independent (can run concurrently):** fetch today's radar image, download the 5-day forecast, and load the local news feed — no task needs another's output, so run them **in parallel** to load the screen faster and keep the UI responsive.
- **Dependent (must stay sequential):** you must **download** the forecast data **before** you can render its graph — a data dependency forces order.
- **Trade-off:** on a multi-core phone the fetches are truly parallel; on a single core they are time-sliced (little speed gain). Shared data (e.g. a cache) needs synchronising to avoid a **race condition**.

Exam tips

- **Apply, don't define** — name the scenario's **independent** tasks and check for **data dependencies** before claiming concurrency.
- **Discuss = both sides** — pair speed/responsiveness benefits with trade-offs (race conditions, deadlock, overhead).
- State that concurrency is **not automatically faster** — single core = time-slicing; some tasks are inherently sequential.
- Use the exact terms **race condition** and **deadlock** correctly to earn AO3 marks.